

Library Lending System

Relational database design, ER diagram, SQL schema, and sample queries

This document designs a relational database for a library lending system, based on the classic problem from *Fundamentals of Database Systems* (Elmasri & Navathe). It separates book titles from physical copies so that availability can be tracked per copy, which is what allows the system to prevent a book from being issued when no copy is free while still preserving a complete loan history.

1. Entities and Design Overview

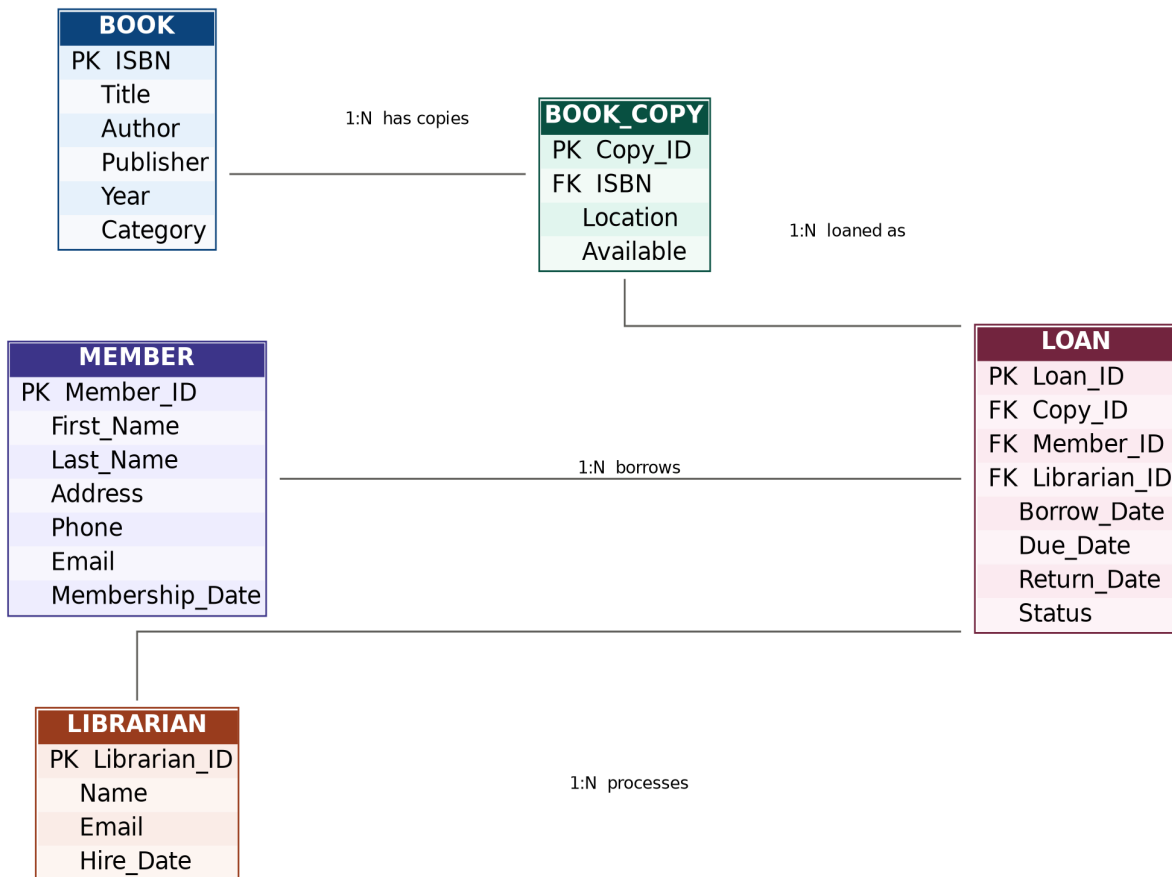
Five tables are used. **BOOK** holds catalog-level (title) information shared by every copy of an edition.

BOOK_COPY represents one physical, borrowable item and carries the availability flag and shelf location.

MEMBER and **LIBRARIAN** hold people records. **LOAN** is the transaction table linking a copy, a member, and the librarian who processed it, and is never deleted -- it doubles as the audit/history log.

- **BOOK** — One row per catalog title/edition (identified by ISBN). Shared by all physical copies of that edition.
- **BOOK_COPY** — One row per physical, lendable item. Holds the Available flag and shelf Location; this is the row that answers “is a copy free right now?”
- **MEMBER** — Library users who can borrow books.
- **LIBRARIAN** — Staff who issue and receive loans.
- **LOAN** — One row per borrowing transaction. Open loans have Return_Date = NULL; closed loans have it filled in. Never deleted, so it serves as loan history.

2. Entity-Relationship Diagram



Relationships: BOOK (1) — (N) BOOK_COPY · BOOK_COPY (1) — (N) LOAN · MEMBER (1) — (N) LOAN · LIBRARIAN (1) — (N) LOAN. A book cannot be issued if its BOOK_COPY.Available is false; the application (or a trigger, see §5) flips this flag on issue and return.

3. Attributes, Primary Keys, and Foreign Keys

BOOK

Attribute	Type	Key	Notes
ISBN	VARCHAR(13)	PK	Shared by every copy of this edition
Title	VARCHAR(255)		NOT NULL
Author	VARCHAR(255)		
Publisher	VARCHAR(150)		
Year	SMALLINT		Year of publication
Category	VARCHAR(80)		Genre / subject

BOOK_COPY

Attribute	Type	Key	Notes
Copy_ID	INT	PK	Auto-increment; identifies one physical item
ISBN	VARCHAR(13)	FK	References BOOK(ISBN)
Location	VARCHAR(50)		Shelf / section
Available	BOOLEAN		NOT NULL, default TRUE

MEMBER

Attribute	Type	Key	Notes
Member_ID	INT	PK	Auto-increment
First_Name	VARCHAR(80)		NOT NULL
Last_Name	VARCHAR(80)		NOT NULL
Address	VARCHAR(255)		
Phone	VARCHAR(20)		
Email	VARCHAR(120)		UNIQUE
Membership_Date	DATE		NOT NULL, default current date

LIBRARIAN

Attribute	Type	Key	Notes
Librarian_ID	INT	PK	Auto-increment
Name	VARCHAR(120)		NOT NULL
Email	VARCHAR(120)		UNIQUE

Hire_Date	DATE		
-----------	------	--	--

LOAN

Attribute	Type	Key	Notes
Loan_ID	INT	PK	Auto-increment
Copy_ID	INT	FK	References BOOK_COPY(Copy_ID)
Member_ID	INT	FK	References MEMBER(Member_ID)
Librarian_ID	INT	FK	References LIBRARIAN(Librarian_ID), nullable
Borrow_Date	DATE		NOT NULL, default current date
Due_Date	DATE		NOT NULL; typically Borrow_Date + 14 days
Return_Date	DATE		NULL while the loan is open
Status	VARCHAR(12)		'borrowed' 'returned' 'overdue'

4. SQL: CREATE TABLE Statements

Written for PostgreSQL syntax (SERIAL / BOOLEAN / CHECK constraints); adjust identity-column syntax for other engines such as MySQL (AUTO_INCREMENT) or SQL Server (IDENTITY).

```
CREATE TABLE BOOK (
    ISBN          VARCHAR(13)  PRIMARY KEY,
    Title         VARCHAR(255) NOT NULL,
    Author        VARCHAR(255),
    Publisher      VARCHAR(150),
    Year          SMALLINT     CHECK (Year > 0),
    Category      VARCHAR(80)
);

CREATE TABLE BOOK_COPY (
    Copy_ID       SERIAL        PRIMARY KEY,
    ISBN          VARCHAR(13)   NOT NULL REFERENCES BOOK(ISBN)
                                ON UPDATE CASCADE ON DELETE RESTRICT,
    Location      VARCHAR(50),
    Available     BOOLEAN       NOT NULL DEFAULT TRUE
);

CREATE TABLE MEMBER (
    Member_ID     SERIAL        PRIMARY KEY,
    First_Name    VARCHAR(80)   NOT NULL,
    Last_Name     VARCHAR(80)   NOT NULL,
    Address       VARCHAR(255),
    Phone         VARCHAR(20),
    Email         VARCHAR(120)  UNIQUE,
    Membership_Date DATE        NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE LIBRARIAN (
    Librarian_ID  SERIAL        PRIMARY KEY,
    Name          VARCHAR(120)  NOT NULL,
    Email         VARCHAR(120)  UNIQUE,
    Hire_Date     DATE
);

CREATE TABLE LOAN (
    Loan_ID       SERIAL        PRIMARY KEY,
    Copy_ID       INT           NOT NULL REFERENCES BOOK_COPY(Copy_ID),
    Member_ID     INT           NOT NULL REFERENCES MEMBER(Member_ID),
    Librarian_ID  INT           REFERENCES LIBRARIAN(Librarian_ID),
    Borrow_Date   DATE          NOT NULL DEFAULT CURRENT_DATE,
    Due_Date      DATE          NOT NULL,
    Return_Date   DATE,
    Status        VARCHAR(12)   NOT NULL DEFAULT 'borrowed'
                                CHECK (Status IN ('borrowed', 'returned', 'overdue')),
    CHECK (Return_Date IS NULL OR Return_Date >= Borrow_Date),
    CHECK (Due_Date >= Borrow_Date)
);

-- At most one OPEN loan per copy at a time (partial unique index)
CREATE UNIQUE INDEX one_open_loan_per_copy
    ON LOAN (Copy_ID)
    WHERE Return_Date IS NULL;
```

5. Keeping Availability Accurate

The one_open_loan_per_copy partial unique index guarantees a copy can never have two simultaneous open loans, which is the core of the “no double-issue” requirement. Two small triggers keep BOOK_COPY.Available in sync automatically, so the application never has to remember to update it by hand:

```
CREATE OR REPLACE FUNCTION mark_copy_unavailable() RETURNS TRIGGER AS $$
BEGIN
    UPDATE BOOK_COPY SET Available = FALSE WHERE Copy_ID = NEW.Copy_ID;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_issue_book
AFTER INSERT ON LOAN
FOR EACH ROW EXECUTE FUNCTION mark_copy_unavailable();

CREATE OR REPLACE FUNCTION mark_copy_available() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.Return_Date IS NOT NULL AND OLD.Return_Date IS NULL THEN
        UPDATE BOOK_COPY SET Available = TRUE WHERE Copy_ID = NEW.Copy_ID;
        NEW.Status := 'returned';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_return_book
BEFORE UPDATE ON LOAN
FOR EACH ROW EXECUTE FUNCTION mark_copy_available();
```

6. Sample Queries

Issue a book (borrow)

```
-- Only succeeds if the copy is currently available
INSERT INTO LOAN (Copy_ID, Member_ID, Librarian_ID, Borrow_Date, Due_Date)
SELECT Copy_ID, 101, 3, CURRENT_DATE, CURRENT_DATE + INTERVAL '14 days'
FROM BOOK_COPY
WHERE Copy_ID = 55 AND Available = TRUE;
-- The AFTER INSERT trigger then sets BOOK_COPY.Available = FALSE
```

Return a book

```
UPDATE LOAN
SET Return_Date = CURRENT_DATE
WHERE Loan_ID = 4821 AND Return_Date IS NULL;
-- The BEFORE UPDATE trigger sets Status = 'returned'
-- and marks BOOK_COPY.Available = TRUE
```

Find overdue books

```
SELECT l.Loan_ID, b.Title, m.First_Name, m.Last_Name, l.Due_Date
FROM LOAN l
JOIN BOOK_COPY bc ON bc.Copy_ID = l.Copy_ID
JOIN BOOK b ON b.ISBN = bc.ISBN
JOIN MEMBER m ON m.Member_ID = l.Member_ID
```

```
WHERE l.Return_Date IS NULL  
      AND l.Due_Date < CURRENT_DATE;
```

Find books currently available

```
SELECT b.ISBN, b.Title, b.Author, bc.Copy_ID, bc.Location  
FROM BOOK b  
JOIN BOOK_COPY bc ON bc.ISBN = b.ISBN  
WHERE bc.Available = TRUE;
```

Borrowing history of a member

```
SELECT l.Loan_ID, b.Title, l.Borrow_Date, l.Due_Date, l.Return_Date, l.Status  
FROM LOAN l  
JOIN BOOK_COPY bc ON bc.Copy_ID = l.Copy_ID  
JOIN BOOK b      ON b.ISBN = bc.ISBN  
WHERE l.Member_ID = 101  
ORDER BY l.Borrow_Date DESC;
```